



Unidad IV

Clase 6

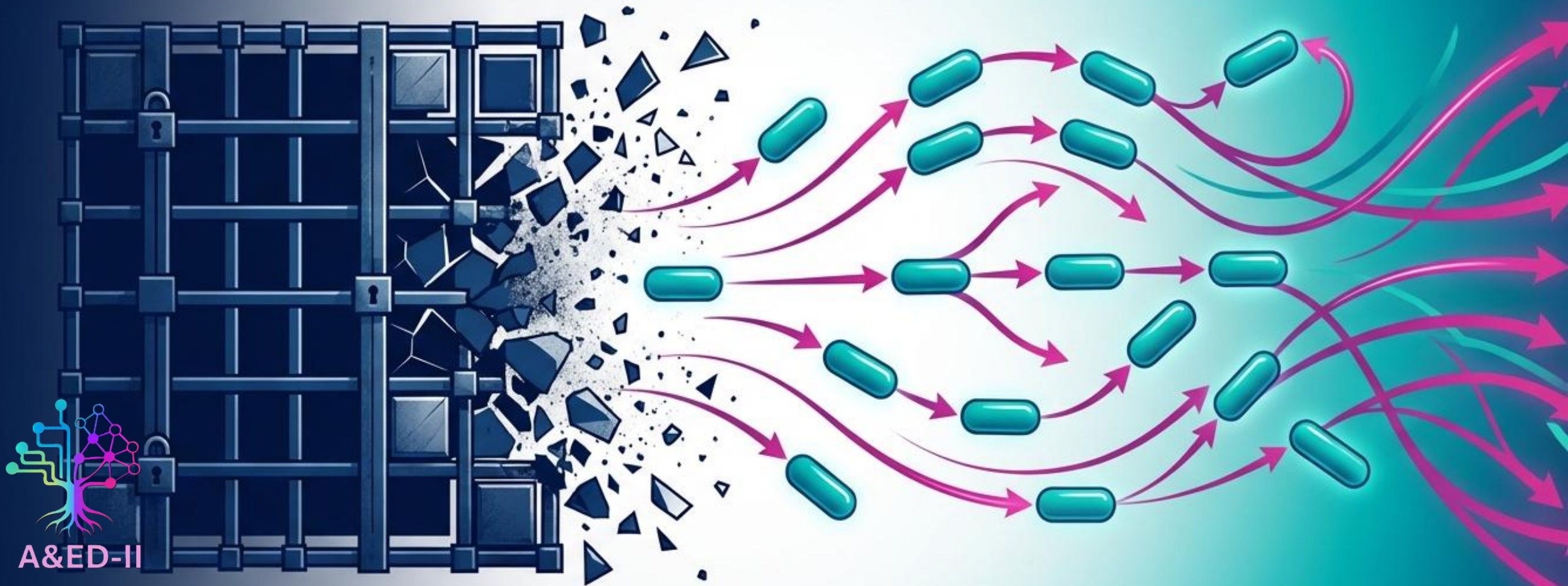
Implementaciones dinámicas

LISTAS DINÁMICAS

*Comprender la diferencia conceptual entre
memoria estática y dinámica.*

Listas dinámicas: cuando el arreglo no alcanza

De la memoria estática a las estructuras infinitas.



¿Qué pasa si el arreglo no alcanza?

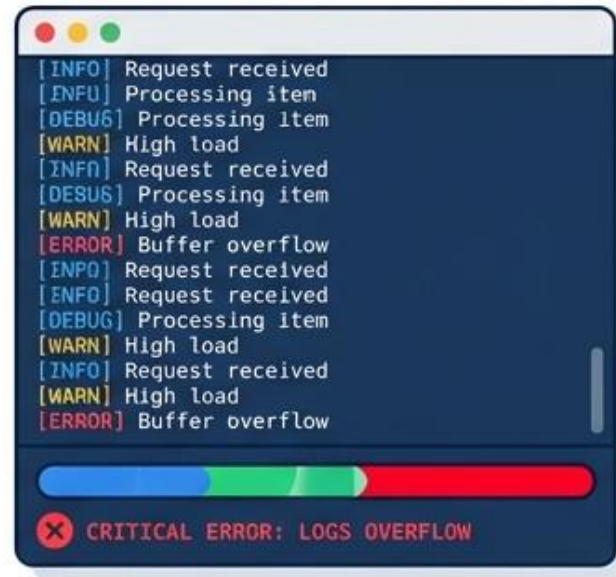
El límite físico de las estructuras estáticas.



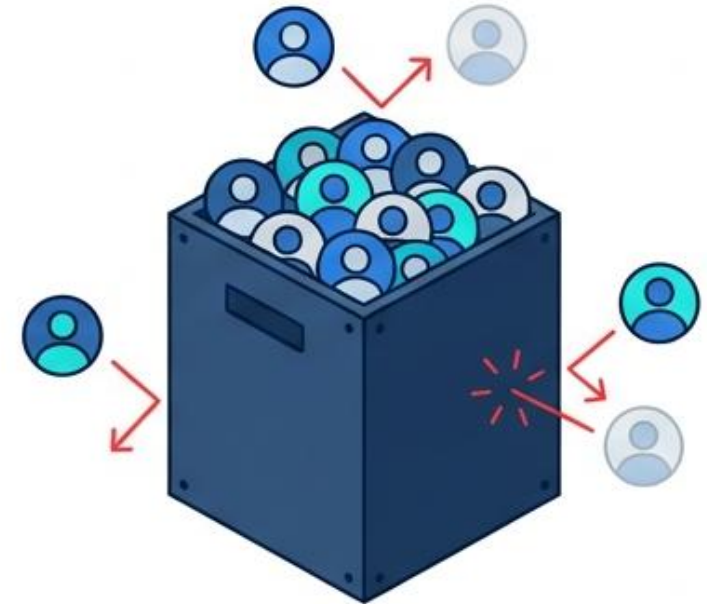
UADE



Carrito saturado



Logs desbordados



Usuarios rechazados

El mundo real no tiene tamaño fijo. Si reservamos `new int[100]`, el usuario 101 rompe el sistema.

El modelo estático: rígido y predecible

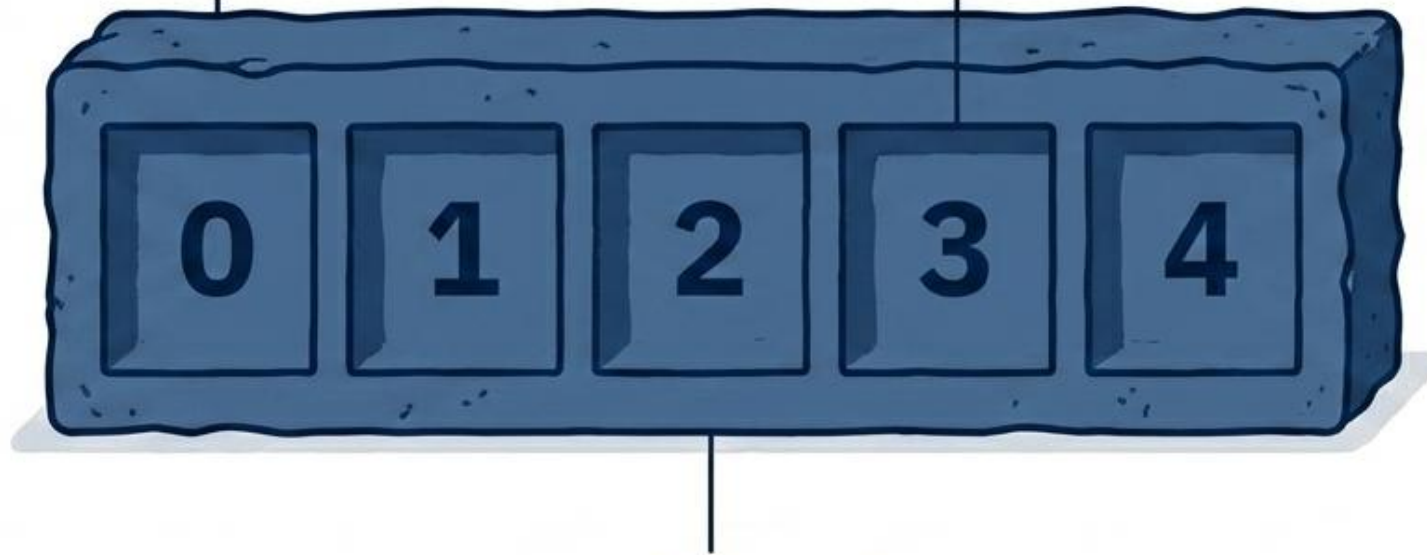
Lo que ya conocemos.

Tamaño Fijo

La memoria se reserva completa al inicio.

Índice

Acceso directo por posición matemática.



Memoria Continua

Los datos viven físicamente uno al lado del otro.

El costo oculto del arreglo

Desperdicio y corrimientos.

Desperdicio



Reservar por las dudas
= **Memoria inactiva.**

Corrimientos



Insertar en el medio
cuesta tiempo lineal: **$O(n)$.**

El salto conceptual: Memoria Dinámica

Pedir espacio solo cuando se necesita.



Memoria bajo
demanda

Reserva gradual:

Crece nodo a nodo,
sin límites
predefinidos.

Dispersión:

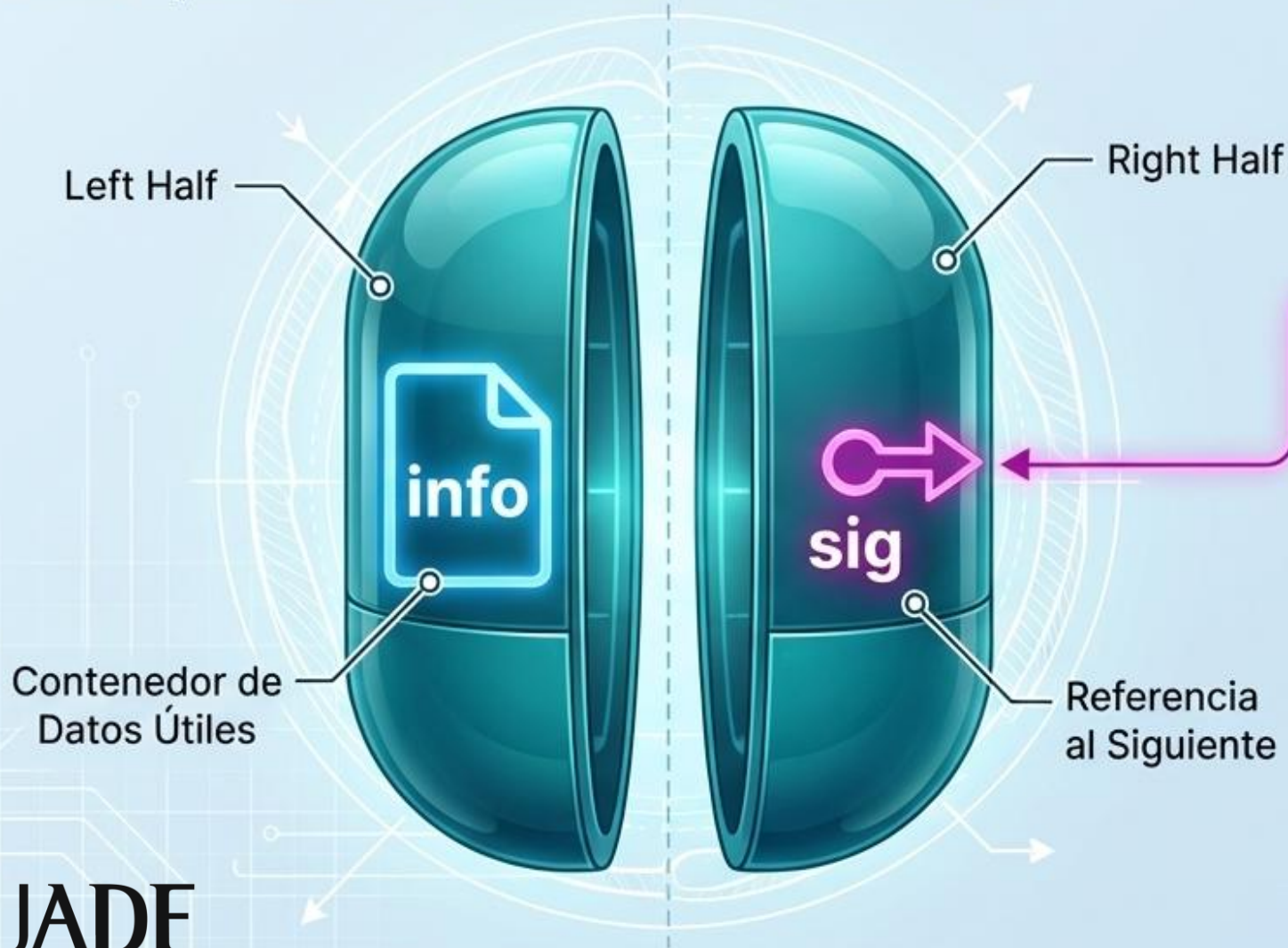
Los elementos ya no
están físicamente
juntos en la memoria.

Conexión:

Se unen mediante
hilos invisibles
(referencias).

Anatomía de un Nodo

El bloque constructor de lo dinámico.



¡ATENCIÓN!

La referencia **`sig`** NO es un índice. Es la dirección exacta en memoria del próximo elemento.

```
class Nodo {  
    int info; // El dato útil  
    Nodo sig; // La referencia al siguiente  
}
```

La Lista Enlazada

Una cadena de referencias.



UADE



primero:

La única puerta de entrada a la estructura. Si se pierde, se pierde toda la lista.

sig:

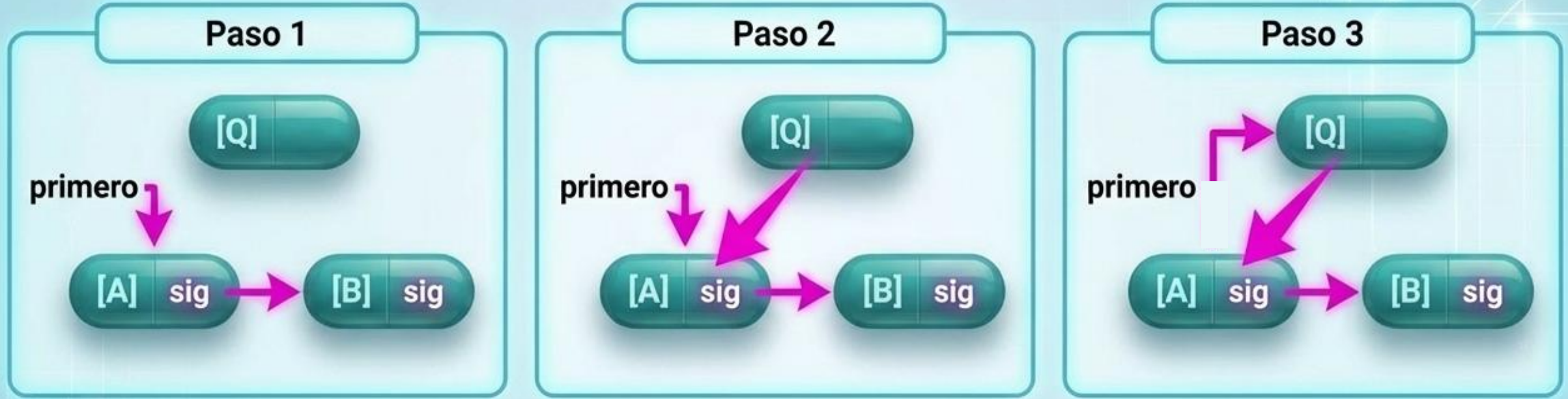
El hilo que mantiene la estructura unida.

null:

La ausencia de un nodo. Marca el final del camino.

Insertar al inicio: Magia en $O(1)$

Reorganizar hilos, no mover datos.



Costo $O(1)$ - ¡Cero corrimientos!

```
Nodo nuevo = new Nodo(x);  
nuevo.sig = primero;  
primero = nuevo;
```

Eliminar al inicio y soltar

El Garbage Collector entra en acción.



UADE

Paso 1

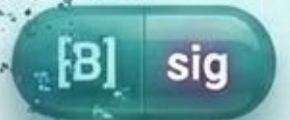
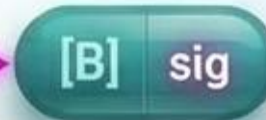
primero



```
primero = primero.sig;
```

Paso 2: Recolección de Basura

primero



En Java, la memoria sin referencias se limpia sola. No hay que **destruir** el nodo manualmente; solo basta con desconectarlo.

El TDA no cambia, el motor sí

Pila Estática vs. Pila Dinámica.



UADE



Estática



Dinámica

El **comportamiento** (TDA) define *qué hace*.
La **implementación** define *cómo lo hace*.

Expandiendo el universo dinámico

Adaptando la cadena a otros comportamientos.



UADE

Cola (FIFO)



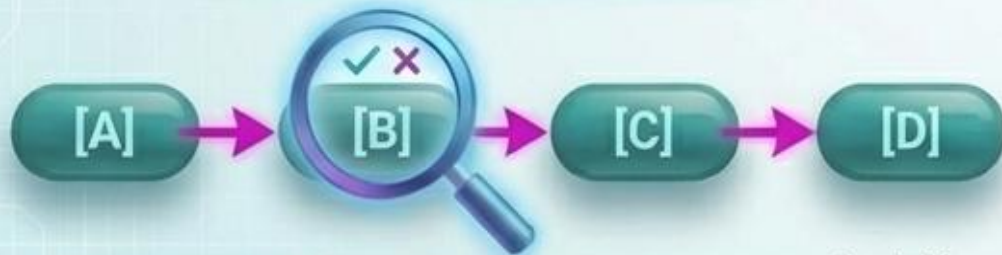
Se guarda una referencia extra al final para acolar en $O(1)$ sin recorrer.

Cola con Prioridad



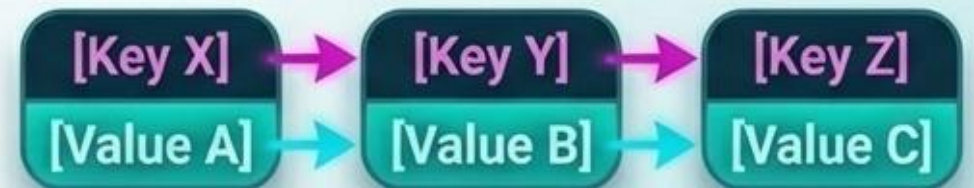
El orden dicta dónde se enlaza el nuevo nodo.

Conjuntos (Sets)



Evita repetidos. Requiere recorrer ($O(n)$) para verificar existencia.

Diccionarios



Pares enlazados. Flexibilidad total de almacenamiento.

El veredicto: ¿Cuándo usar qué?

Toda decisión arquitectónica tiene un costo.



UADE

El Poder Dinámico



Flexibilidad total:

Crece y decrece según la necesidad.



Inserciones eficientes:

$O(1)$ al inicio, cero desplazamientos.



Eficiencia espacial:

Cero memoria inactiva (desperdicio).



El Costo



Sobrecarga de memoria:

Cada nodo exige espacio extra para la referencia (sig').



Complejidad conceptual:

Riesgo de perder referencias (NullPointerException).



No hay acceso directo:

Para llegar al nodo 50, hay que pasar por los 49 anteriores.

Usa **arreglos** cuando sepas la cantidad exacta y necesites acceso directo.
Usa **listas dinámicas** cuando el **tamaño sea impredecible** y los datos cambien constantemente.

IMPLEMENTACIÓN DE PILA DINÁMICA CON NODOS

Pila Dinámica (PilaLD) basada en nodos enlazados. Crece según necesidad y gestiona LIFO mediante el cambio de referencias de memoria, evitando el costo de mover elementos.

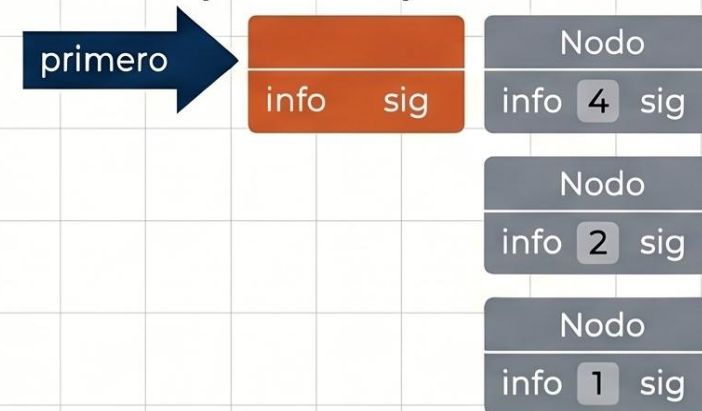
```
class Nodo {  
    int info;  
    Nodo sig;  
}  
  
public class PilaLD implements PilaTDA {  
    Nodo primero;  
  
    public void InicializarPila() {  
        primero = null;  
    }  
  
    public void Apilar(int x) {  
        Nodo nuevo = new Nodo();  
        nuevo.info = x;  
        nuevo.sig = primero;  
        primero = nuevo;  
    }  
  
    public void Desapilar() {  
        primero = primero.sig;  
    }  
  
    public boolean PilaVacía() {  
        return primero == null;  
    }  
  
    public int Tope() {  
        return primero.info;  
    }  
}
```

LÓGICA DE FUNCIONAMIENTO



El nuevo nodo apunta al "primero";
"primero" pasa a ser el nuevo.

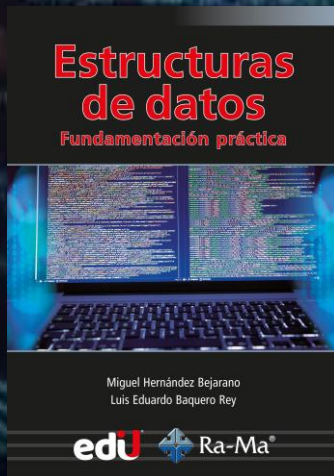
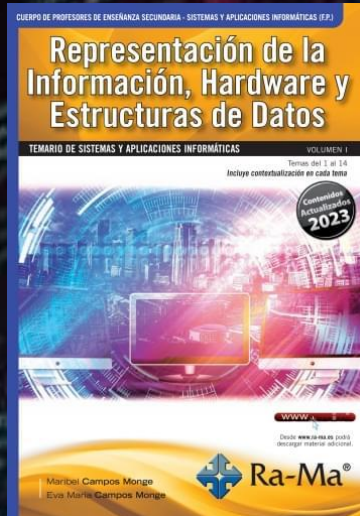
ELIMINAR (DESAPILAR)



KEY FINDING EFICIENCIA POR REFERENCIAS

No existen corrimientos
físicos de datos, solo
cambios en los enlaces
entre nodos.

👉 ¿CÓMO LO HARÍAMOS CON UN DICCIONARIO SIMPLE?



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



1 Parcial

Unidad 1: Marco conceptual

Unidad II: de la abstracción a la implementación

TDA Pila, TDA Cola, TDA Cola con Prioridad

TDA Conjunto, Diccionario simple y múltiple

Combinar estructuras para resolver problemas reales

Contrato, Firma y Código